TALLER DE METODOLOGÍAS ÁGILES · 2026-2 · TERCER CORTE

Punto de control del proyecto final

Programar en pareja con una persona o con un agente de IA: qué se gana, qué se pierde y qué aprendizaje desaparece

TEMA DE EXPOSICIÓN

3 · Pair programming

PRÁCTICA QUE ASISTE EL AGENTE

Desarrollo de una tarea en pair programming, con el agente en cuatro configuraciones

CASO BASE

Notitas: un tablero kanban de notas adhesivas

PAREJA

Ana Sofía Arango Yanza · Juan José Arias Chacua

PROFESOR

César Jesús Pardo Calvache, Ph.D.

VERSIÓN

v11 · noviembre de 2026

REPOSITORIO

github.com/Sofii141/notitas · las referencias [R1]…[R20] de este documento están enlazadas en la sección 11

1 Introducción

Queremos saber qué cambia cuando un agente de IA entra al pair programming, y si depende del rol que ocupa: revisar, escribir, acompañar a una pareja o trabajar con otro agente.

1. **La práctica.** En el *pair programming*, una práctica que nació con Extreme Programming (Beck, 1999; Williams y Kessler, 2002), dos personas programan juntas: una escribe el código (el *conductor*) y la otra revisa y guía (el *navegante*). Lo compararemos con cuatro configuraciones en las que un agente de IA ocupa uno de esos roles (escenarios E1 a E4, sección 5).

2. **El material (caso base).** Construimos **Notitas**, un tablero kanban de notas adhesivas con un tablero visual y **24 reglas de backend sin programar**. Cada regla es una tarea, de diferentes tamaños, con sus criterios de aceptación y sus pruebas automáticas [R1] [R2].

3. **Antes del agente (línea base).** Programaremos 5 de esas tareas en pareja, sin IA, cronometrando y grabando. Ya hicimos 3, una de cada tamaño, compartiendo pantalla en una llamada (sección 7); las 2 que faltan las haremos con VS Code Live Share, una herramienta hecha para programar en pareja.

4. **Con el agente (dos ciclos de una semana).** Programaremos 12 tareas más en cuatro escenarios que cambian quién escribe, quién revisa y cuántos agentes hay (sección 5), y registraremos cada uso del agente en una bitácora.

5. **La comparación.** Mediremos en todos los escenarios lo mismo: tiempo, errores que se escapan, retrabajo y cuánto entendemos del código que quedó (sección 4).

CASO BASE

24 tareas

10 de ellas difíciles a propósito

FORMAS DE TRABAJAR

5

la línea base (los dos, sin IA) y 4 escenarios con agente

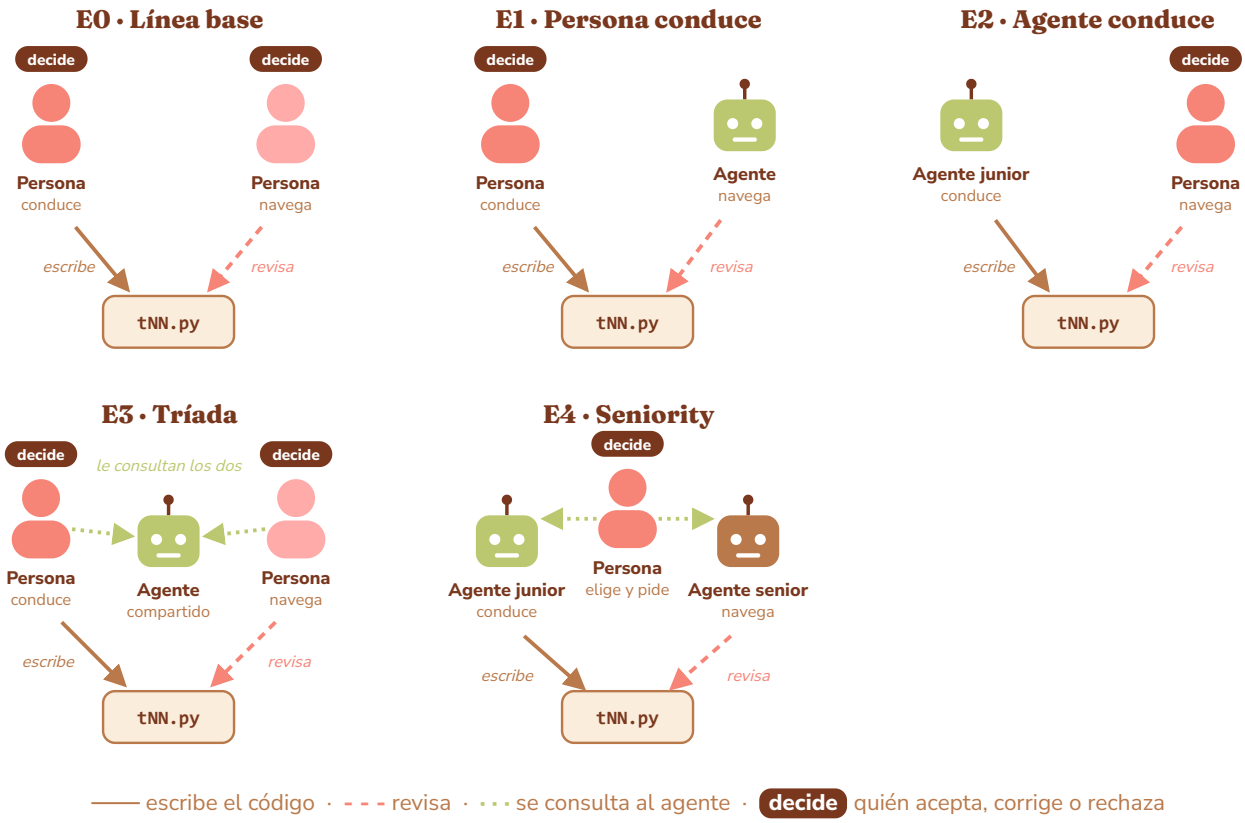
TAREAS QUE MEDIREMOS

17

5 en la línea base y 3 en cada escenario con agente

Las cinco formas de trabajar

Cada dibujo muestra quién escribe el código (línea continua), quién lo revisa (línea punteada) y quién decide si se acepta. E0 es la línea base, los dos sin IA. E1 a E4 son los escenarios con agente: en E1 la persona escribe y el agente revisa; en E2 al revés; en E3 la pareja trabaja con un agente que ven los dos; y en E4 un agente escribe, otro revisa y la persona decide. La sección 5 los explica uno por uno.



escribe el código · revisa · se consulta al agente · **decide** quién acepta, corrige o rechaza

Términos que usamos en el documento

Conductor y navegante	Los dos roles del pair programming: el conductor escribe el código y el navegante lo revisa, piensa en casos borde y guía.
Línea base	La medición de cómo trabajamos hoy, en pareja y sin IA, antes de usar el agente. Es el escenario E0.
Agente	Un programa de IA que lee el código y propone o escribe cambios. Usamos dos: OpenCode con el modelo DeepSeek y Claude Code con el modelo Claude.
Agente junior / senior	El junior escribe código; el senior revisa y pregunta, sin escribir. La persona siempre decide.
Escenario	Una forma de trabajar: quién escribe el código, quién lo revisa y si hay agente. Son cinco, de E0 (línea base) a E4.
Ciclo	Una semana de trabajo real con el agente, sobre al menos 5 tareas. Hay dos.
Tamaño S, M, L	Tarea pequeña (unos 15–25 min), mediana (25–45 min) o grande (45–70 min), según su complejidad (sección 3).
Pruebas visibles	Pruebas automáticas que se ven mientras se programa y dicen cuándo la tarea está terminada.
Pruebas ocultas	Pruebas que nadie ve hasta terminar. Cada una que falla es un error que se escapó.
Retrabajo	El tiempo que se tarda corrigiendo lo que encontraron las pruebas ocultas.
Codewalk	Cinco preguntas sobre el código, respondidas sin mirarlo, para medir cuánto se entendió (0 a 5).
Pronóstico	Cuántos minutos creemos que tomará una tarea, anotado después de leer el enunciado y antes de empezar.
M1 a M5	Las cinco mediciones de cada tarea: tiempo, errores escapados, retrabajo, codewalk y pronóstico (sección 4).

Sorteo y semilla	Un programa reparte las tareas y su orden; con el mismo número (la semilla) siempre da lo mismo, y así cualquiera puede comprobarlo.
Caso congelado	El caso base marcado en el repositorio antes de medir: desde ahí no se cambian las tareas ni las pruebas.
Bitácora	La hoja de cálculo donde se anota cada uso del agente y si se aceptó, se corrigió o se rechazó, y por qué.
Live Share	Extensión de VS Code para que dos personas programen en el mismo código, cada una desde su computador.

2 Qué esperamos ver

Antes de medir, revisamos estudios publicados para saber qué es razonable esperar. Los usamos como punto de comparación, no como hipótesis que vayamos a probar.

Con 5 tareas en la línea base y 3 en cada escenario, nuestros datos no alcanzan para afirmar que el agente causa una diferencia, ni para generalizar a otros equipos. Lo que sí podremos hacer es describir si nuestros datos van en la misma dirección que esos estudios o en otra, y buscar por qué en la bitácora y en las grabaciones. La literatura revisada por pares sobre programar con IA apunta a tres cosas, y cada una corresponde a una de nuestras mediciones:

Qué esperamos	Por qué lo esperamos
Que el tiempo baje en las tareas sencillas, pero no en las difíciles	Con IA los estudiantes terminaron un 35 % más rápido (Shihab et al., 2025), pero el efecto de programar en pareja depende de la complejidad de la tarea (Hannay et al., 2009; Arisholm et al., 2007).
Que se escapen más errores y haya más retrabajo	Las sugerencias de la IA se aceptan con menos escrutinio que las de un compañero (Welter et al., 2025), y con IA se escribe código menos seguro creyendo que es más seguro (Perry et al., 2023), y una parte del código que genera tiene errores (Moradi Dakhel et al., 2023).
Que se entienda menos el código, sobre todo el compañero que no participó	En pareja se transfiere conocimiento (Plonka et al., 2015); con IA, los estudiantes no entendían por qué funcionaban las sugerencias (Shihab et al., 2025). Aunque otro estudio no encontró pérdida de aprendizaje (Kazemitabaar et al., 2023).

Qué esperamos de cada escenario

Escenario	Qué esperamos y por qué
E1 la persona escribe y el agente revisa	Lo más parecido a la línea base: con IA en el lugar del compañero se han visto resultados comparables a los de parejas humanas (Fan et al., 2025), aunque con algo menos de comprensión del compañero, que no participó (Plonka et al., 2015).
E2 el agente escribe y la persona revisa	El más rápido, pero con la comprensión más baja: quien no escribe entiende menos y puede creer que entiende (Prather et al., 2024; Lee et al., 2025). Es el escenario con menos estudios previos.
E3 dos personas y un agente compartido	Menos dependencia del agente que trabajando solo, porque el compañero ve lo que se le pide (Daryanto et al., 2026); la IA para sintaxis y conceptos, y el compañero para ideas (Lyu et al., 2025).
E4 junior escribe, senior revisa, persona decide	Menos errores escapados que E2, porque un agente revisa al otro (Qian et al., 2024; Hong et al., 2024), pero más tiempo.

Son expectativas, no conclusiones. Si los datos dicen otra cosa, lo reportaremos así y lo trataremos como un resultado, no como un error: también es útil saber que algo no se cumplió en nuestro caso. La forma de presentarlo está en la sección 8.

3 El caso base: Notitas

Notitas es un tablero de notas adhesivas con tres columnas (*Por hacer · En curso · Hecho*). El tablero visual ya está hecho; lo que falta son las reglas que lo hacen funcionar.

Cómo está hecho

El repositorio tiene dos partes [R1]. El **frontend** [R7] es el tablero, con arrastrar y soltar y animaciones; ya está terminado y **no se mide**. El **backend** [R3] tiene 24 funciones vacías: cada tarea consiste en escribir la lógica de una de ellas. En cuanto una función se implementa correctamente, el tablero empieza a usarla.

Así, el cronómetro mide solo el desarrollo de la tarea, no la configuración, y todas las tareas se pueden comparar entre sí. Ninguno de los dos integrantes de la pareja había programado antes estas reglas.

Qué lo hace representativo y difícil

- Son las reglas típicas de un tablero real: validar datos, mover notas entre columnas, buscar y filtrar, fechas de vencimiento, deshacer, exportar.
- **10 de las 24 son difíciles a propósito**: concurrencia, zonas horarias, texto con tildes y otros caracteres, inyección de código en el contenido, copias que comparten datos sin querer y reglas que parecen obvias pero no lo son.
- Cada tarea tiene pruebas visibles y ocultas [R4]: **41 visibles** y **72 ocultas**. Comprobamos que las 113 fallan con las funciones vacías y pasan con soluciones de referencia, que no están en el repositorio.

Tamaño de las tareas y en qué se basa

Tamaño	Cuántas	Tiempo estimado	Criterio
pequeña (S)	10	15–25 min	Una función que valida o transforma un dato, con reglas independientes entre sí
mediana (M)	12	25–45 min	Varias reglas que interactúan, fechas o zonas horarias, o una clase que guarda estado
grande (L)	2	45–70 min	Concurrencia o un algoritmo de varios pasos con casos de conflicto

La estimación la propuso Claude Code al diseñar las tareas, con el criterio de la tabla: cuántas reglas tiene que combinar la función y si involucra fechas, estado guardado o concurrencia. Nosotros revisamos ese criterio en general, con un ejemplo, pero no leímos los criterios de cada tarea, para no conocer de antemano lo que íbamos a programar. No es un dato del estudio: solo sirve para que cada condición reciba una mezcla parecida de tareas. El número de criterios de aceptación no distingue los tamaños (todas tienen entre 3 y 5), por eso no lo usamos. La duración real la da el cronómetro, y si un tamaño no se cumple, lo reportaremos.

Las 24 tareas [R2]

ID	Tarea	Tamaño	Por qué es difícil
`t01`	Validar el título de una nota	pequeña	

ID	Tarea	Tamaño	Por qué es difícil
t02	Color de la nota	pequeña	
t03	Transiciones entre columnas	mediana	Difícil Muchas combinaciones y una regla que parece obvia pero no lo es (no se salta "En curso")
t04	Límite WIP de "En curso"	mediana	Difícil Reordenar dentro de la misma columna no debe contar como una nota más
t05	Insertar en una posición	mediana	
t06	Filtrar por etiquetas	pequeña	
t07	Buscar sin tildes ni mayúsculas	mediana	Difícil Unicode: tildes, ñ y letras como ß que no se resuelven con lower()
t08	Estado de vencimiento	mediana	Difícil Zona horaria: a las 10 p. m. de Bogotá ya es el día siguiente en UTC
t09	Normalizar etiquetas	pequeña	
t10	Qué notas se archivan	pequeña	
t11	Deshacer movimientos	mediana	
t12	Dos personas mueven la misma nota	grande	Difícil Concurrencia: dos llamadas al mismo tiempo con la misma versión; solo una puede ganar
t13	Exportar e importar el tablero	mediana	
t14	Contenido seguro (sin inyectar HTML)	mediana	Difícil Trampa de seguridad: hay que escapar antes de dar formato, no después
t15	Resumen por columna	pequeña	
t16	Ordenar por prioridad y fecha	pequeña	
t17	Paginar el listado	pequeña	
t18	Iniciales del responsable	pequeña	
t19	Progreso de la checklist	mediana	
t20	Duplicar una nota	pequeña	Difícil Trampa de mutabilidad: una copia superficial comparte las listas con la original
t21	Fusionar cambios de dos dispositivos	grande	Difícil Fusión a tres bandas, empates y fechas con zonas horarias distintas
t22	Notas bloqueadas	mediana	

ID	Tarea	Tamaño	Por qué es difícil
`t23`	Tiempo de ciclo	mediana	Difícil La nota puede ir y volver entre columnas, y los eventos pueden llegar desordenados
`t24`	Límite de notas por minuto	mediana	Difícil Ventana deslizante: los bordes y los intentos rechazados

4 Qué medimos (GQM)

Objetivo: analizar el desarrollo de una tarea con un agente en distintos roles (escenarios E1 a E4), frente al pair programming humano (E0), para saber si cambian el tiempo, los errores que se escapan, el retrabajo y la comprensión del código.

Usamos el enfoque GQM (Basili et al., 1994): de un objetivo salen preguntas, y de cada pregunta una métrica. Son cinco métricas **directas** [R10]: cada una es un número que se anota tal cual, sin fórmulas. Las tres primeras son las que el enunciado sugiere para el tema 3 (tiempo, retrabajo y errores detectados a tiempo); la cuarta responde la pregunta del tema (qué aprendizaje desaparece) y la quinta muestra si la sensación de velocidad engaña.

Pregunta	Métrica	Cómo se toma	Qué NO representa
¿Se termina más rápido?	**M1** Minutos hasta que pasan las pruebas visibles	Hora de inicio y de fin, menos pausas	La productividad en general
¿Se detectan los errores a tiempo?	**M2** Pruebas ocultas que fallan al decir "terminado"	Se corren las ocultas de esa tarea	Defectos de diseño o de legibilidad
¿Hay retrabajo?	**M3** Minutos de corrección después de "terminado"	Cronómetro aparte (0 si no hubo)	El tamaño de la corrección
¿Qué aprendizaje se pierde?	**M4** Codewalk de 0 a 5, al autor y al compañero	5 preguntas sin mirar el código [R11]	La habilidad general de la persona
¿Nos engaña la sensación de velocidad?	**M5** Pronóstico en minutos, tras leer el enunciado y antes de empezar	Se compara con M1	El resultado real

Las 5 preguntas del codewalk

Son las mismas para todas las tareas y en todos los escenarios, porque preguntan por la comprensión y no por una tarea o una forma de trabajar en particular: así la nota se puede comparar entre escenarios. Solo cambian la X y la Y, que están fijadas por tarea para que sean iguales en todos los escenarios [R11].

1. ¿Qué hace la función, en una frase?
2. ¿Qué caso borde maneja y dónde?
3. ¿Por qué se eligió esta forma y no otra?
4. Si mañana falla con la entrada X, ¿dónde mirarías primero?
5. ¿Qué cambiarías si el requisito cambia en Y?

Para ver si se pierde pensamiento crítico no agregamos métricas: comparamos el codewalk (M4) entre escenarios y contamos cuántas salidas del agente se aceptan sin modificar. En los ciclos, la bitácora registra además cada uso del agente con su escenario, qué agente fue, su decisión (aceptar, corregir o rechazar), su razón y su responsable [R9].

5 Cómo mediremos

	Línea base (E0)	Ciclos 1 y 2 (E1 a E4)
Quién trabaja	Los dos en llamada, sin IA, con VS Code Live Share: en cada tarea uno escribe y el otro guía; el conductor se alterna por tarea	Según el escenario: una persona con uno o dos agentes (E1, E2, E4) o las dos personas con un agente compartido (E3)
Cuántas tareas	5: 2 pequeñas, 2 medianas y 1 grande (3 ya medidas)	12, 3 por escenario: E1 y E2 en el ciclo 1, E3 y E4 en el ciclo 2. Cada persona decide 7 tareas en total. Los 3 casos de otra pareja se sumarán al ciclo 2 cuando los recibamos
Evidencia	Grabación de pantalla y audio (con Live Share cuando hay dos personas; cuando se trabaja solo con agentes basta con la pantalla y la voz, porque la conversación con el agente queda además en la ficha y en la bitácora), captura del cronómetro, un commit al empezar, al terminar y al corregir cada tarea, la hoja de medición y la hoja de sesiones [R9]; en los ciclos, además, una fila de bitácora por cada uso del agente	

Los escenarios

En la asesoría del 06/11 recibimos el consejo de explorar varias formas de trabajar, porque el pair programming está cambiando con la IA: el agente y la persona en las dos direcciones, dos personas más un agente, y agentes con roles de seniority. Con eso diseñamos cuatro escenarios. Todos miden la misma práctica con las mismas cinco métricas; solo cambia quién escribe, quién revisa y quién decide. El dibujo de la sección 1 muestra cada uno.

Escenario	Escribe	Revisa	Qué pregunta responde	Tareas
E0 Línea base	Persona	Persona	Cómo trabajamos hoy, sin IA	t06, t03, t12, t15, t07
E1 Persona conduce, agente navega	Persona	Agente copiloto	¿Qué pasa si el agente reemplaza al compañero que revisa?	t01, t23, t21
E2 Agente conduce, persona navega	Agente junior	Persona	La otra dirección: ¿qué pasa si el agente escribe y la persona revisa? E1 frente a E2 es la comparación bidireccional.	t02, t22, t19
E3 Tríada: dos personas y un agente	Persona	Persona, con un agente compartido	¿Qué le suma el agente a una pareja humana? Es la comparación más directa con la línea base.	t16, t04, t14
E4 Agentes con seniority	Agente junior	Agente senior; la persona decide	¿Repartir roles entre agentes da mejor código y deja a la persona pensando más?	t09, t18, t11

E1 frente a E2 muestra si importa quién escribe (la comparación en las dos direcciones). E3 frente a E0 muestra qué le suma un agente a una pareja. E4 frente a E2 muestra si un segundo agente que revisa ayuda. Los escenarios son un diseño propio, inspirado en estudios de roles en pair programming (Williams et al., 2000; Chong y Hurlbutt, 2007), de parejas con IA (Lyu et al., 2025; Daryanto et al., 2026) y de agentes con roles (Qian et al., 2024; Hong et al., 2024).

Roles

Conductor y navegante son funciones, no niveles de experiencia: en parejas reales los dos trabajan al mismo nivel y pesa más quién sabe más (Chong y Hurlbutt, 2007; Bryant et al., 2008). Por eso la seniority la definimos en los agentes.

Rol	Qué hace	Qué no hace
Conductor	Escribe el código y explica lo que hace	No decide solo si la tarea terminó
Navegante	Revisa, piensa en casos borde y vigila los criterios de aceptación	No escribe en el editor
Decisor (siempre una persona)	Acepta, corrige o rechaza lo que propone el agente, con su razón; corre las pruebas ocultas	No delega esa decisión
Agente junior	Escribe el código que se le pide y declara sus supuestos	No elige el enfoque ni da la tarea por terminada
Agente senior	Revisa, pregunta y señala riesgos	No escribe la solución

Quién hace cada tarea

Persona	Línea base	E1	E2	E3 (conduce)	E4
Ana Sofía	t03, t15	t01, t23	t02	t16, t14	t11
Juan José	t06, t12, t07	t21	t22, t19	t04	t09, t18

Las tareas salen del sorteo con semilla 1234: E1 y E2 usan las del ciclo 1, con la persona asignada; E3 usa las primeras de línea base que quedaron libres, porque se compara con la línea base; E4 usa las primeras del ciclo 2. Las demás quedan de repuesto, en orden.

Por qué 5 tareas de línea base y 3 por escenario

- **Cubren los tres tamaños.** El efecto del pair programming cambia con la complejidad de la tarea (Hannay et al., 2009; Arisholm et al., 2007), así que la línea base tiene S, M y L.
- **Es un estudio exploratorio.** Con 5 o con 10 casos los resultados son descriptivos, no causales (Kitchenham et al., 2017; Wohlin et al., 2012). Siguiendo el consejo de la asesoría, preferimos explorar cuatro configuraciones con 3 casos cada una que una sola con muchos casos.
- **El costo es real.** Una tarea de línea base tomó entre 24 y 76 minutos, más el codewalk y el registro. Con 24 tareas en total, cada caso de línea base es uno menos para los escenarios.
- **Grabación.** Las 3 primeras sesiones las hicimos compartiendo pantalla en una llamada. Con ellas probamos el procedimiento completo (la ficha, el codewalk y el registro) y lo ajustamos: por ejemplo, ahora guardamos una captura del cronómetro y una carpeta de respuestas por tarea. A partir de los consejos de la asesoría, las 2 que faltan y el E3 se harán con VS Code Live Share, una herramienta hecha para programar en pareja, y se sigue grabando pantalla y audio, como en los estudios que analizan sesiones reales de pair programming (Plonka et al., 2015).

REGLAS PARA NO DEJAR DE PENSAR

En todos los escenarios con agente: (1) 2 minutos para anotar el enfoque propio antes de abrir el agente; (2) no se acepta una línea que no se pueda explicar sin el agente; (3) el senior responde primero con preguntas y no con código; (4) las pruebas ocultas las corre siempre una persona; (5) cada uno decide en E1 y en E2, y los dos conducen en E3. Responden al riesgo de aceptar lo que dice la IA sin pensarlo (Lee et al., 2025; Prather et al., 2024).

Pasos por tarea

1. Leer el enunciado, máximo 2 minutos y sin programar, y anotar cuántos minutos creemos que tomará (M5).
2. Iniciar el cronómetro.
3. Anotar el enfoque propio (con agente) y programar según el escenario, con ayuda solo de las pruebas visibles.
4. Cuando pasan, declarar "terminado" y parar el cronómetro (M1).
5. Correr las pruebas ocultas y anotar cuántas fallan (M2). Después corregimos siempre hasta que pasen, sin límite de tiempo y con cronómetro aparte: ese tiempo es retrabajo (M3).
6. Codewalk (M4): autor y compañero responden por escrito, al mismo tiempo y sin mirar el código; después cada uno califica al otro con el código abierto. Las respuestas se guardan.
7. Llenar la hoja de medición en ese momento, no al final del día.

CONTROLES PARA QUE LA COMPARACIÓN SEA JUSTA

- **Sorteo** [R5] [R6]: un programa reparte las tareas y sortea su orden, de modo que cada condición reciba una mezcla parecida de tamaños y de tareas difíciles. Se puede repetir con la misma semilla y da lo mismo. Los escenarios toman las tareas en ese mismo orden, sin elegirlas a mano.
- **Mismas ayudas en todas las tareas:** documentación de Python y búsquedas puntuales en la web; ninguna otra IA aparte de los agentes de cada escenario.
- **Caso congelado** antes de medir: desde ese momento no se cambian las tareas ni las pruebas.
- Si alguien ve las pruebas ocultas antes de tiempo, esa tarea se descarta y se usa una de repuesto.
- Los agentes trabajan en una copia del proyecto **sin** las pruebas ocultas y con permisos limitados [R8].

6 Limitaciones que declaramos

Lo que este diseño no permite afirmar, para que los resultados se lean con cuidado.

LO QUE ESTE DISEÑO NO PUEDE MOSTRAR

- El codewalk lo califican los mismos participantes, cada uno al otro, sin un evaluador externo. Para comprobarlo, al final una IA que no usamos en los ciclos (ni DeepSeek ni Claude) recalifica las respuestas guardadas, sin datos de autor ni de escenario, y reportamos la coincidencia; la nota oficial sigue siendo la de la pareja, y el código aún puede delatar la condición.
- El tiempo de cada tarea (M1, sección 4) no incluye la lectura inicial del enunciado (máximo 2 minutos), que es igual en todos los escenarios.
- Las pruebas se derivaron de los criterios de aceptación y de casos borde, y se validaron contra funciones vacías y soluciones de referencia; no se midió su cobertura.
- En la línea base, cada tarea la escribe una sola persona: no hay cambio de rol dentro de la tarea.
- Son 5 tareas de línea base y 3 por escenario: alcanzan para describir tendencias, no para comparar con estadística. En la asesoría del 06/11 acordamos que la cantidad puede ser flexible si se justifica (sección 5).
- Las 3 primeras sesiones de línea base se hicieron compartiendo pantalla en una llamada, y las 2 restantes se harán con Live Share. La forma de trabajar es la misma (uno escribe y el otro guía); cambia la herramienta, y se anota en la hoja de sesiones.
- Los escenarios son un diseño propio, a partir de los consejos de la asesoría del 06/11 e inspirado en estudios publicados; ninguno de esos estudios trae esta combinación tal cual.
- **Es un caso diseñado para el experimento**, no el backlog de un proyecto en curso: los resultados no se generalizan a un sistema real grande.
- Las tareas, las pruebas y el tablero se generaron con Claude Code. Para que el caso no quede "a la medida" del agente: el agente que escribe código es **otro modelo** (DeepSeek), nadie ve las pruebas ocultas antes de terminar, ningún agente tiene acceso a ellas, y los 3 casos de otra pareja se incorporarán cuando los recibamos. En E4 se usa Claude Opus 5.5 como revisor: no escribe código ni ve las ocultas, pero es de la misma familia con que se construyó el caso, y lo declaramos.
- Las tareas son reglas aisladas: no medimos el costo de integrarlas con un sistema más grande.

7 Primeros resultados: línea base

Lo que sigue lo genera un script a partir de la bitácora **[R9]** **[R15]**, con lo medido en las sesiones cronometradas y grabadas. No se escribe a mano. Son las 3 tareas medidas hasta ahora; faltan t15 y t07. Son primeros resultados: por sí solos no permiten conclusiones, pero muestran que el procedimiento funciona y cómo se verán los datos.

Condiciones de la medición

Fecha (dd/mm/aaaa)	Hora inicio	Hora fin	Participantes	Lugar o canal	Herramientas (editor, Python, pytest)	IA usada (ninguna / herramienta + modelo exacto)	Semilla del sorteo	Interrupciones o incidentes
02/11/2026	14:42	16:17	Ana Sofía Arango, Juan José Arias	Llamada	Python, pytest	ninguna	1234	Sin pausas. Por tiempo, una tarea de cada tamaño primero. Video completo, incluye lo previo al cronómetro; hora fin = fin de la grabación
02/11/2026	18:44	19:22	Ana Sofía Arango, Juan José Arias	Llamada	Python, pytest	ninguna	1234	Sin pausas. Hora fin = fin de la grabación
02/11/2026	19:25	21:27	Ana Sofía Arango, Juan José Arias	Llamada	Python, pytest	Claude (Claude Code) solo para aclarar el CA5; sin solución	1234	Consulta del CA5 a Claude. Pausas no anotadas [N]. Hora inicio y fin = grabación

Resultados por tarea (n = 3)

Tarea	Tam.	Difícil	M5 pronóstico	M1 minutos	M2 ocultas que fallan	M3 retrabajo (min)	M4 codewalk autor / comp.
`t03` Transiciones entre columnas	M	Sí	35	24	0/3	0	5 / 5
`t06` Filtrar por etiquetas	S		35	47	0/3	0	5 / 5
`t12` Dos personas mueven la misma nota	L	Sí	60	54	1/3	22	5 / 5

Resumen por tamaño

M1 y M4: mediana [mínimo–máximo]. Son tendencias, no causas.

Tamaño	n	M1 minutos	M2 ocultas que fallan	M3 tareas con retrabajo	M4 codewalk autor / comp.	M5 tardaron más de lo pronosticado
S	1	47	0 de 3	0 de 1	5 / 5	1 de 1
M	1	24	0 de 3	0 de 1	5 / 5	0 de 1
L	1	54	1 de 3	1 de 1 (22 min)	5 / 5	0 de 1
Total	3	47 [24–54]	1 de 9	1 de 3 (22 min)	5 [5–5] / 5 [5–5]	1 de 3

8 Cómo presentaremos los resultados

Con pocas tareas por escenario, mostraremos todos los datos y diremos qué no demuestran [R14].

Qué reportamos	Ejemplo	Fuente
Mediana [mínimo–máximo] de minutos y del codewalk, por escenario y por tamaño	"Tareas medianas: 32 min [21–48] en pareja y 25 min [18–60] con el agente"	Kitchenham et al., 2017
Conteos de errores escapados y de retrabajo	"Se escaparon 4 de 30 errores; 3 de 10 tareas tuvieron retrabajo"	Arisholm et al., 2007
Pronóstico frente a tiempo real	"En 7 de 12 tareas tardamos más de lo que creíamos"	Perry et al., 2023
Decisiones de la bitácora	"De 48 usos: 20 aceptados, 21 corregidos y 7 rechazados"	Enunciado del curso
Sin afirmar causas, y en cada cifra, qué no demuestra	"No prueba que el agente cause la diferencia: son pocas tareas y distintas en cada condición"	Kitchenham et al., 2002; Wohlin et al., 2012
Una cifra recalculada por la otra persona	El script, la hoja de cálculo y un cálculo a mano deben coincidir	—

Así presentaremos todos los datos al final: la línea base y los cuatro escenarios, con la bitácora completa. Una de las cifras la recalculará a mano la otra persona.

9 Especificación de los agentes

Cada escenario usa uno o dos agentes, cada uno con su especificación:

Escenario	Agente	Spec
E1 la persona escribe, el agente revisa	OpenCode · DeepSeek V4.1 Flash, como navegante	Copiloto v1
E2 el agente escribe, la persona revisa	OpenCode · DeepSeek V4.1 Flash, como conductor	Junior v1
E3 dos personas y un agente	OpenCode · DeepSeek V4.1 Flash, consultado por los dos	Copiloto v1
E4 junior escribe, senior revisa	OpenCode · DeepSeek V4.1 Flash (junior) y Claude Code · Claude Opus 5.5 (senior)	Junior v1 y Senior v1

Copiloto v1 · navegante (E1 y E3) [R12]

Los 8 campos son los que pide el enunciado del proyecto. La columna *Fundamento* dice en qué investigación o norma se apoya cada decisión.

Campo	Contenido	Fundamento
Rol y alcance	Hace de navegante en el desarrollo de *una* tarea de Notitas: propone un enfoque, el código y los casos borde que conviene probar. La persona escribe y decide.	Roles de conductor y navegante (Williams et al., 2000; Bryant et al., 2008)
Entradas	Solo el enunciado de la tarea, su archivo del backend y sus pruebas visibles.	Compartir solo lo necesario (NIST AI 600-1, 2024)
Salidas	En formato fijo: un plan de 3 a 6 pasos, el cambio propuesto en el código, los supuestos que hizo y los casos borde que cree que faltan.	El enunciado: sin formato definido, una salida no se puede evaluar
Herramientas y permisos	**OpenCode** con el modelo gratuito **DeepSeek V4.1 Flash**, el mismo para los dos todo el ciclo (el modelo y la versión se anotan). Trabaja en una copia del proyecto sin las pruebas ocultas; tiene prohibido leerlas, cambiar las pruebas, guardar o subir cambios al repositorio, salir de la carpeta y buscar en internet, y para todo lo demás pide permiso [R8].	Mínimo privilegio (NIST AI 600-1, 2024). No es de la familia Claude, con la que se construyó el caso
Prohibiciones	No guardar ni subir cambios al repositorio. No modificar pruebas ni otros archivos. No leer las pruebas ocultas. No desactivar ni saltarse pruebas. No inventar requisitos. No manejar credenciales ni datos personales.	Validez de la medición (Ko et al., 2015); código inseguro con IA (Perry et al., 2023)
Criterio de validación	Se acepta solo si pasan las pruebas visibles, cumple cada criterio de aceptación y quien conduce puede explicar cada línea sin mirar al agente.	Menos escrutinio con IA (Welter et al., 2025); no entender las sugerencias (Shihab et al., 2025)

Campo	Contenido	Fundamento
Punto de control humano	Antes de guardar cada cambio, quien conduce decide aceptar, corregir o rechazar y lo anota en la bitácora con su razón. Las pruebas ocultas las corre la persona. Después viene el codewalk.	Human-in-the-loop (NIST AI 600-1, 2024; sesión 3 del curso)
Versión	v1. Cada cambio da una versión nueva, con el dato de la bitácora que la motivó.	El enunciado

La spec Copiloto v1 se usa en E1 y en E3. Para E2 y E4 hay dos specs más, con los mismos 8 campos y su fundamento [R16]:

Campo	Junior v1 (E2 y E4)	Senior v1 (E4)
Rol y alcance	Conductor: escribe el código de la tarea en cambios pequeños y explica cada uno	Navegante mentor: revisa lo que escribe el junior, pregunta y recomienda; no escribe código
Entradas	Enunciado, archivo de la tarea y pruebas visibles	Lo mismo, con el código que dejó el junior
Salidas	Un cambio por vez, con qué cambió, supuestos y casos borde	Hasta 3 preguntas, riesgos con su línea, casos borde y recomendación aceptar, corregir o rechazar
Herramientas y permisos	OpenCode con DeepSeek V4.1 Flash; pide permiso antes de cada edición	Claude Code con Claude Opus 5.5, en solo lectura: no edita, no corre comandos, no busca en internet ni lee las ocultas [R8]
Prohibiciones	Las mismas del copiloto, y no repetir igual un cambio rechazado	No escribir la solución ni decidir por la persona
Criterio de validación	La persona lo leyó y lo puede explicar sin el agente, pasan las visibles y cumple los criterios	Una observación se usa solo si la persona la comprueba contra el código o un criterio
Punto de control humano	La persona decide cada salida (aceptar, corregir o rechazar) con su razón en la bitácora, y corre las pruebas ocultas	
Versión	v1. Cada cambio da una versión nueva, con el dato de la bitácora que la motivó	

Cómo se articulan en E4

- **Qué se pasan:** nada directamente. El junior escribe en el archivo de la tarea y el senior lo lee; lo que va del senior al junior lo elige y lo escribe la persona.
- **Última palabra:** la persona. Ningún agente acepta, corre las pruebas ocultas ni guarda cambios.
- **Si no coinciden:** decide la persona con las pruebas visibles y los criterios de aceptación, y anota por qué.
- **Misma fuente:** los dos leen el mismo enunciado y el mismo código, así que coincidir no prueba que estén bien. Por eso son modelos de familias distintas, el senior ve el código y no el razonamiento del junior, y la validación final la hacen las pruebas ocultas, que ninguno ve.

10 Calendario

Caso base
Congelado y sorteado

Línea base
5 tareas en pareja, sin IA, con Live Share

Ciclo 1
E1 y E2: 6 tareas, cada uno con el agente

Ciclo 2
E3 y E4: 6 tareas, specs ajustadas y 3 casos de otra pareja

16–18 nov
Reporte, diapositivas y exposición

11 Fuentes

Todas revisadas por pares, salvo la norma de NIST y los dos libros, que se indican. Los DOI se verificaron en Crossref y OpenAlex. La lista completa, con la base de datos de cada una, está en **[R13]**.

- Arisholm, E., Gallis, H., Dybå, T., & Sjøberg, D. I. K. (2007). Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE Transactions on Software Engineering, 33*(2), 65–86. doi:10.1109/TSE.2007.17
- Basili, V., Caldiera, G., & Rombach, H. D. (1994). The Goal Question Metric approach. En *Encyclopedia of Software Engineering*. Wiley.
- Beck, K. (1999). *Extreme programming explained: Embrace change*. Addison-Wesley. (Libro.)
- Bryant, S., Romero, P., & du Boulay, B. (2008). Pair programming and the mysterious role of the navigator. *International Journal of Human-Computer Studies, 66*(7), 519–529. doi:10.1016/j.ijhcs.2007.03.005
- Chong, J., & Hurlbutt, T. (2007). The social dynamics of pair programming. *ICSE 2007*, 354–363. doi:10.1109/ICSE.2007.87
- Daryanto, T. et al. (2026). Human-human-AI triadic programming: Uncovering the role of AI agent and the value of human partner in collaborative learning. *CHI '26*. arXiv:2601.12134
- Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance. *International Journal of STEM Education, 12*, 16. doi:10.1186/s40594-025-00537-3
- Hannay, J. E., Dybå, T., Arisholm, E., & Sjøberg, D. I. K. (2009). The effectiveness of pair programming: A meta-analysis. *Information and Software Technology, 51*(7), 1110–1122. doi:10.1016/j.infsof.2009.02.001
- Hong, S. et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR 2024*.
- Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. *CHI 2023*, 1–23. doi:10.1145/3544548.3580919
- Kitchenham, B. et al. (2002). Preliminary guidelines for empirical research in software engineering. *IEEE TSE, 28*(8), 721–734. doi:10.1109/TSE.2002.1027796
- Kitchenham, B., Madeyski, L., Budgen, D., Keung, J., Brereton, P., Charters, S., Gibbs, S., & Pohthong, A. (2017). Robust statistical methods for empirical software engineering. *Empirical Software Engineering, 22*(2), 579–630. doi:10.1007/s10664-016-9437-5
- Ko, A. J., LaToza, T. D., & Burnett, M. M. (2015). A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering, 20*(1), 110–141. doi:10.1007/s10664-013-9279-3
- Lee, H.-P. et al. (2025). The impact of generative AI on critical thinking. *CHI 2025*, 1–22. doi:10.1145/3706598.3713778
- Lyu, W., Wang, Y., Sun, Y., & Zhang, Y. (2025). Will your next pair programming partner be human? An empirical evaluation of generative AI as a collaborative teammate in a semester-long classroom setting. *L@S '25*, 83–94. arXiv:2505.08119
- Microsoft. (s. f.). Coedit, follow, and focus in a collaboration session in Visual Studio Code. *Visual Studio Live Share*. (Documentación.) https://learn.microsoft.com/en-us/visualstudio/liveshare/use/coedit-follow-focus-visual-studio-code
- Moradi Dakhel, A. et al. (2023). GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software, 203*, 111734. doi:10.1016/j.jss.2023.111734
- NIST (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile* (NIST AI 600-1). doi:10.6028/NIST.AI.600-1

- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? *ACM CCS 2023*, 2785–2799. doi:10.1145/3576915.3623157
- Plonka, L., Sharp, H., van der Linden, J., & Dittrich, Y. (2015). Knowledge transfer in pair programming: An in-depth analysis. *International Journal of Human-Computer Studies, 73*, 66–78. doi:10.1016/j.ijhcs.2014.09.001
- Prather, J. et al. (2024). The widening gap: The benefits and harms of generative AI for novice programmers. *ICER 2024*. arXiv:2405.17739
- Qian, C. et al. (2024). ChatDev: Communicative agents for software development. *ACL 2024*. arXiv:2307.07924
- Shihab, M. I. H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., & Mulanda, B. (2025). The effects of GitHub Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. *ICER 2025*, 407–420. doi:10.1145/3702652.3744219
- Welter, A., Schneider, N., Dick, T., Weis, K., Tinnes, C., & Wyrich, M. (2025). An empirical study of knowledge transfer in AI pair programming. *ASE 2025*, 166–177. doi:10.1109/ASE63991.2025.00022
- Williams, L., Kessler, R. R., Cunningham, W., & Jeffries, R. (2000). Strengthening the case for pair programming. *IEEE Software, 17*(4), 19–25. doi:10.1109/52.854064
- Williams, L., & Kessler, R. (2002). *Pair programming illuminated*. Addison-Wesley. (Libro.)
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. doi:10.1007/978-3-642-29044-2

12 Dónde está cada cosa en el repositorio

Las referencias [R1]…[R20] del texto enlazan aquí. No adjuntamos archivos aparte: todo está en el repositorio, con su historial de versiones. El código está en la rama `main` y los documentos en la rama `documentacion`.

Ref.	Qué es	Enlace
R1	Repositorio del proyecto (código del caso base)	rama main
R2	Enunciado y criterios de aceptación de las 24 tareas	TAREAS.md
R3	Código de las tareas (donde se programa la lógica)	backend/notitas/
R4	Pruebas visibles y ocultas	backend/tests/
R5	Sorteo de las tareas	sorteo.py
R6	Resultado del sorteo y registro de sesiones	registro/
R7	Tablero (frontend, no se mide) y servidor	frontend/ y backend/servidor.py
R8	Configuración y permisos de los agentes	agente/ (opencode.json y claude_senior/)
R9	Bitácora y hoja de medición	31102026_01_Bitacora_v3.xlsx
R10	GQM	06112026_01_GQM_v4.md
R11	Preguntas del codewalk con su X y su Y por tarea	31102026_02_Codewalk_Preguntas_v1.md
R12	Especificación del agente v1 (con fundamento e historial)	31102026_01_Spec_Copiloto_v1.md
R13	Fuentes revisadas por pares, con DOI	06112026_01_Fuentes_v3.md
R14	Cómo medimos y cómo presentaremos los resultados	31102026_01_Como_Medir_y_Presentar_v3.md
R15	Script que resume la bitácora	resumen_medicion.py
R16	Specs Junior v1 y Senior v1, articulación y prompts	06112026_01_Specs_Escenarios_v1.pdf
R17	Guía de escenarios: división, preparación y reglas	06112026_01_Guia_Escenarios_v1.pdf
R18	Carpetas de cada escenario, con su guía, fichas y codewalk	2_Escenarios_con_agente/
R19	Guía de la línea base con Live Share	06112026_01_Guia_Linea_Base_Pareja_v10.pdf
R20	Bitácora de uso de IA: qué hicimos con IA, qué verificamos y qué corregimos nosotros	08112026_01_Bitacora_IA_v1.pdf

13 Declaración de uso de IA

Herramienta	Para qué	Qué verificamos nosotros
Claude Code	Leer los enunciados; proponer la estructura; generar las 24 tareas con sus pruebas y soluciones de referencia, el backend y el tablero; buscar fuentes y verificar sus DOI en Crossref y OpenAlex; revisar la coherencia de los documentos con un segundo agente revisor; proponer los escenarios, los roles y la herramienta de grabación después de la asesoría; hacer una revisión preliminar, no ciega, de los codewalk; maquetar este documento.	Hicimos el sorteo (semilla 1234) y programamos las tareas de la línea base. Revisamos el criterio general de los tamaños, sin leer los criterios de cada tarea para no anticipar las soluciones. Estamos leyendo las fuentes sobre pair programming con una ruta de lectura repartida entre los dos: Beck (1999), Williams et al. (2000) y Williams y Kessler (2002), Hannay et al. (2009), Bryant et al. (2008) con Chong y Hurlbutt (2007), Plonka et al. (2015), y Lyu et al. (2025) con Daryanto et al. (2026); al terminar, cada uno le explicará al otro lo que leyó. Leímos este documento completo y le hicimos correcciones. Todas nuestras sugerencias, decisiones y correcciones están en la bitácora de IA [R20].

En los ciclos, Claude Opus 5.5 es además el agente senior de E4, que se mide como parte del estudio. Ningún dato de línea base ni de bitácora fue generado por IA: esos datos salen solo de nuestras sesiones cronometradas y grabadas.